SIMATS ENGINEERING

ASSESSMENT TOOL 3 – Assignment 2

COURSE NAME: STATISTICS WITH R PROGRAMMING **COURSE CODE: ITA04**

COURSE OUTCOME COVERED

CO3: *Construct and manipulate data frames using reshaping, merging, and file I/O operations in R to prepare structured datasets for analysis. (BL3, BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks: 100**

Annexure

Assignment 2

Code of Conduct:

I **K. Navadeep Reddy** ***(192311305)*** *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K. Navadeep

Assignment 2:

- **Generate 5 Questions for your Capstone Project module**
- **Apply the formulae and use the relevant graph to visualize it**
- **Upload the file in Github link**
- **Save the 5 Questions and Gihub link in the Excell sheet**

Hospital Readmission Analysis to Identify Factors Associated with Patient Readmission

Module-1: Patient Data Collection and Preprocessing

Question-1: What percentage of values is missing in each attribute of the dataset, and which attributes should be removed or imputed?

Formula Used and Explanation

Missing Percentage (%) = (Number of missing values in a column ÷ Total number of records) × 100

Data collection from hospital systems is rarely complete. In this dataset the symbol "?" represents a missing value, so the first step is to read the file with na.strings so that R treats "?" as NA. The formula above is then applied to every column using colSums(is.na(data)).

Columns with a very high missing percentage (such as weight, payer_code and medical_specialty) carry too little information to be reliable and are removed. Columns with only a small missing percentage (such as race) are retained and the gaps are labelled as "Unknown", so that no patient records are lost.

A bar chart is used because it compares the missing percentage of each attribute side by side and makes the worst columns easy to identify.

R Program

```
# Q1 - Missing value analysis
library(dplyr)
library(ggplot2)

data <- read.csv("diabetic_data.csv", na.strings = c("?", ""),
                 stringsAsFactors = FALSE)
cat("Rows:", nrow(data), " Columns:", ncol(data), "\n")

missing_pct <- colSums(is.na(data)) / nrow(data) * 100
missing_df <- data.frame(Variable = names(missing_pct),
                         Missing_Percent = round(missing_pct, 2),
                         row.names = NULL) %>%
  filter(Missing_Percent > 0) %>%
  arrange(desc(Missing_Percent))
print(missing_df)

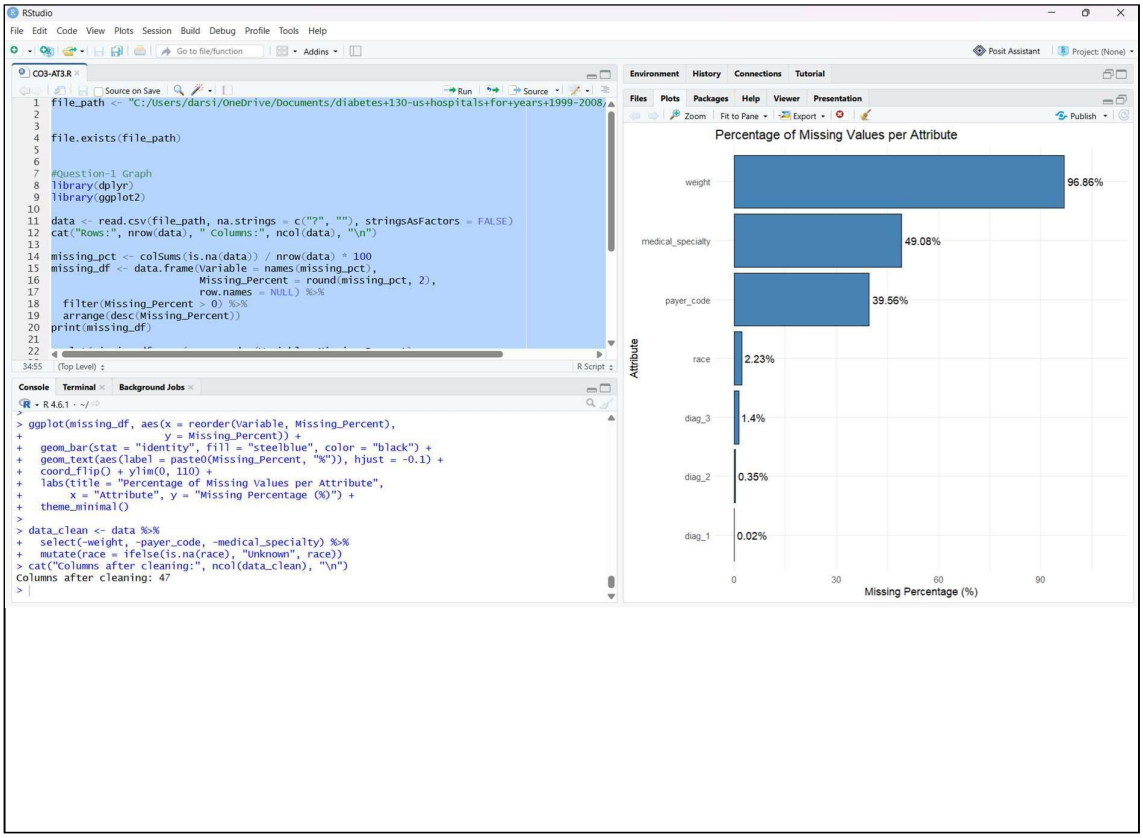
# Graph: horizontal bar chart
ggplot(missing_df, aes(x = reorder(Variable, Missing_Percent),
                       y = Missing_Percent)) +
  geom_bar(stat = "identity", fill = "steelblue", color = "black") +
  geom_text(aes(label = paste0(Missing_Percent, "%")), hjust = -0.1) +
  coord_flip() + ylim(0, 110) +
  labs(title = "Percentage of Missing Values per Attribute",
       x = "Attribute", y = "Missing Percentage (%)") +
  theme_minimal()

# Treatment: drop heavily missing columns, label race as Unknown
data_clean <- data %>%
  select(-weight, -payer_code, -medical_specialty) %>%
  mutate(race = ifelse(is.na(race), "Unknown", race))
cat("Columns after cleaning:", ncol(data_clean), "\n")
```

Graph for Visualization

Graph type: Horizontal bar chart. Each bar shows the missing percentage of one attribute, sorted from highest to lowest.

Output



Question-2: How many repeat encounters exist for the same patient, and how can the dataset be reduced to one independent record per patient?

Formula Used and Explanation

$$\textit{Repeat Encounter Percentage (\%)} = ((N - U) \div N) \times 100$$

where N = total encounters and U = unique patients (patient_nbr)

The dataset is stored at encounter level, so one patient can appear many times. If all rows are used for model building, the same patient is counted repeatedly and the records are no longer independent, which makes the results over-optimistic.

The program counts total encounters (N) and distinct patients (U), applies the formula, and plots how many encounters each patient has. To prepare the data, only the first encounter of each patient is kept. Records where the patient expired or was sent to hospice (discharge_disposition_id = 11, 13, 14, 19, 20, 21) are also removed, because these patients cannot be readmitted.

R Program

```
# Q2 - Duplicate / repeat encounter analysis
library(dplyr)
library(ggplot2)

data <- read.csv("diabetic_data.csv", na.strings = c("?", ""),
                 stringsAsFactors = FALSE)

N <- nrow(data)                      # total encounters
U <- n_distinct(data$patient_nbr)  # unique patients
repeat_pct <- (N - U) / N * 100
cat("Total encounters :", N, "\n")
cat("Unique patients  :", U, "\n")
cat("Repeat encounters:", round(repeat_pct, 2), "%\n")
cat("Duplicate encounter IDs:", sum(duplicated(data$encounter_id)), "\n")

enc_per_patient <- data %>% count(patient_nbr, name = "encounters")

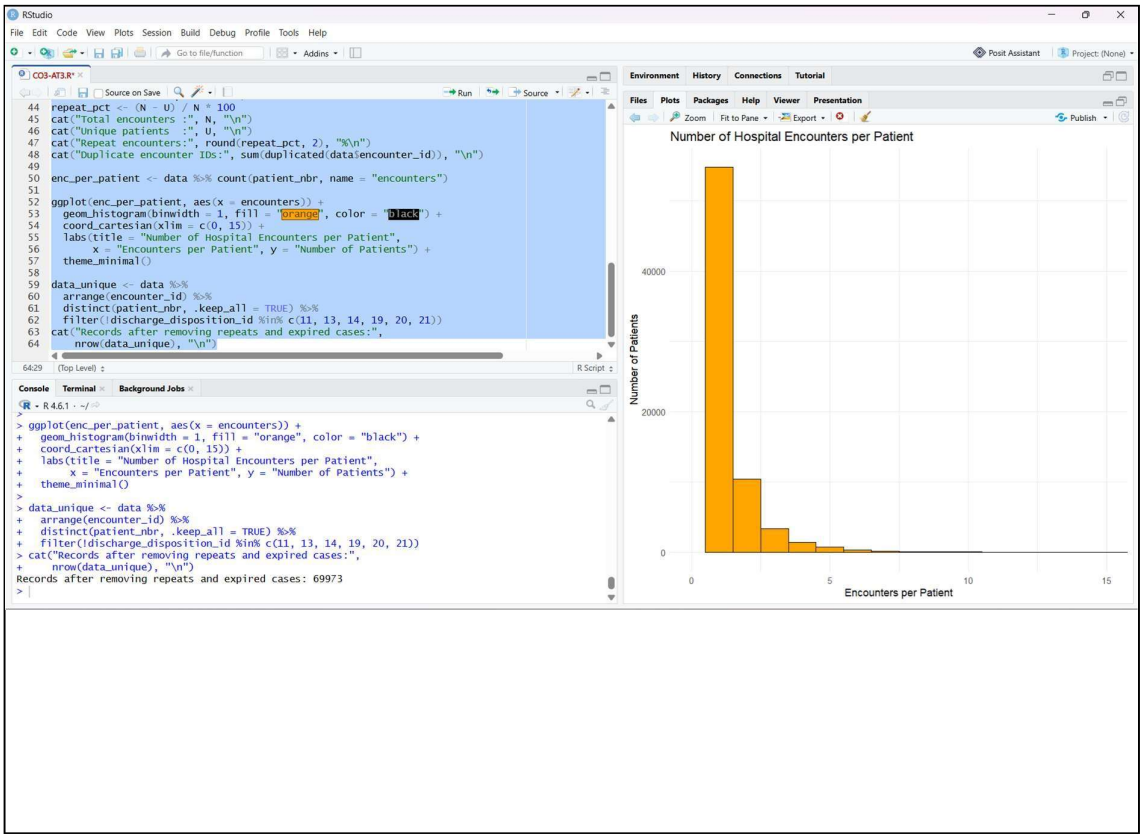
# Graph: histogram
ggplot(enc_per_patient, aes(x = encounters)) +
  geom_histogram(binwidth = 1, fill = "orange", color = "black") +
  coord_cartesian(xlim = c(0, 15)) +
  labs(title = "Number of Hospital Encounters per Patient",
       x = "Encounters per Patient", y = "Number of Patients") +
  theme_minimal()

# Keep first encounter of each patient; remove expired/hospice cases
data_unique <- data %>%
  arrange(encounter_id) %>%
  distinct(patient_nbr, .keep_all = TRUE) %>%
  filter(!discharge_disposition_id %in% c(11, 13, 14, 19, 20, 21))
cat("Records after removing repeats and expired cases:",
    nrow(data_unique), "\n")
```

Graph for Visualization

Graph type: Histogram. The x-axis shows the number of encounters per patient and the y-axis shows the number of patients.

Output


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
CO3-AT3.R*
Source on Save
Run
Source
repeat_pct <- (N - U) / N * 100
cat("Total encounters :", N, "\n")
cat("Unique patients :", U, "\n")
cat("Repeat encounters:", round(repeat_pct, 2), "%\n")
cat("Duplicate encounter IDs:", sum(duplicated(data$encounter_id)), "\n")
enc_per_patient <- data %>% count(patient_nbr, name = "encounters")
ggplot(enc_per_patient, aes(x = encounters)) +
geom_histogram(binwidth = 1, fill = "orange", color = "black") +
coord_cartesian(xlim = c(0, 15)) +
labs(title = "Number of Hospital Encounters per Patient",
x = "Encounters per Patient", y = "Number of Patients") +
theme_minimal()
data_unique <- data %>%
arrange(encounter_id) %>%
distinct(patient_nbr, .keep_all = TRUE) %>%
filter(!discharge_disposition_id %in% c(11, 13, 14, 19, 20, 21))
cat("Records after removing repeats and expired cases:",
nrow(data_unique), "\n")
64:29 (Top Level) R Script
Console Terminal Background Jobs
R 4.6.1 · ~/
> ggplot(enc_per_patient, aes(x = encounters)) +
+ geom_histogram(binwidth = 1, fill = "orange", color = "black") +
+ coord_cartesian(xlim = c(0, 15)) +
+ labs(title = "Number of Hospital Encounters per Patient",
+ x = "Encounters per Patient", y = "Number of Patients") +
+ theme_minimal()
> data_unique <- data %>%
+ arrange(encounter_id) %>%
+ distinct(patient_nbr, .keep_all = TRUE) %>%
+ filter(!discharge_disposition_id %in% c(11, 13, 14, 19, 20, 21))
> cat("Records after removing repeats and expired cases:",
+ nrow(data_unique), "\n")
Records after removing repeats and expired cases: 69973
Environment History Connections Tutorial
Files Plots Packages Help Viewer Presentation
Zoom Fit to Pane Export Publish
Number of Hospital Encounters per Patient
Number of Patients
40000
20000
0
0
5
10
15
Encounters per Patient

Question-3: Which numerical attributes contain outliers, and how many records fall outside the acceptable range?

Formula Used and Explanation

$$IQR = Q3 - Q1$$

$$Lower\ Fence = Q1 - 1.5 \times IQR \qquad Upper\ Fence = Q3 + 1.5 \times IQR$$

$$A\ value\ is\ an\ outlier\ if\ x < Lower\ Fence\ or\ x > Upper\ Fence$$

Q1 is the 25th percentile and Q3 is the 75th percentile of a variable. The interquartile range (IQR) measures the spread of the middle 50% of the data. Any value beyond 1.5 times the IQR from the quartiles is flagged as an outlier.

The program calculates the fences and the number of outliers for time_in_hospital, num_lab_procedures and num_medications. Because these are real clinical values, they are not simply deleted; instead, extreme values are capped at the upper fence (winsorizing) so that they do not dominate the analysis.

A box plot is used because it shows the median, quartiles and outliers of each variable.

R Program

```
# Q3 - Outlier detection using the IQR method
library(dplyr)
library(tidyr)
library(ggplot2)

data <- read.csv("diabetic_data.csv", na.strings = c("?", ""),
                 stringsAsFactors = FALSE)
num_vars <- c("time_in_hospital", "num_lab_procedures", "num_medications")

outlier_table <- do.call(rbind, lapply(num_vars, function(v) {
  x     <- data[[v]]
  Q1    <- unname(quantile(x, 0.25))
  Q3    <- unname(quantile(x, 0.75))
  IQR_v <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR_v
  upper <- Q3 + 1.5 * IQR_v
  n_out <- sum(x < lower | x > upper)
  data.frame(Variable = v, Q1 = Q1, Q3 = Q3, IQR = IQR_v,
             Lower = lower, Upper = upper, Outliers = n_out,
             Outlier_Percent = round(n_out / length(x) * 100, 2))
}))
print(outlier_table)
# Graph: box plots (one panel per variable)
long_df <- data %>%
  select(all_of(num_vars)) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Value")

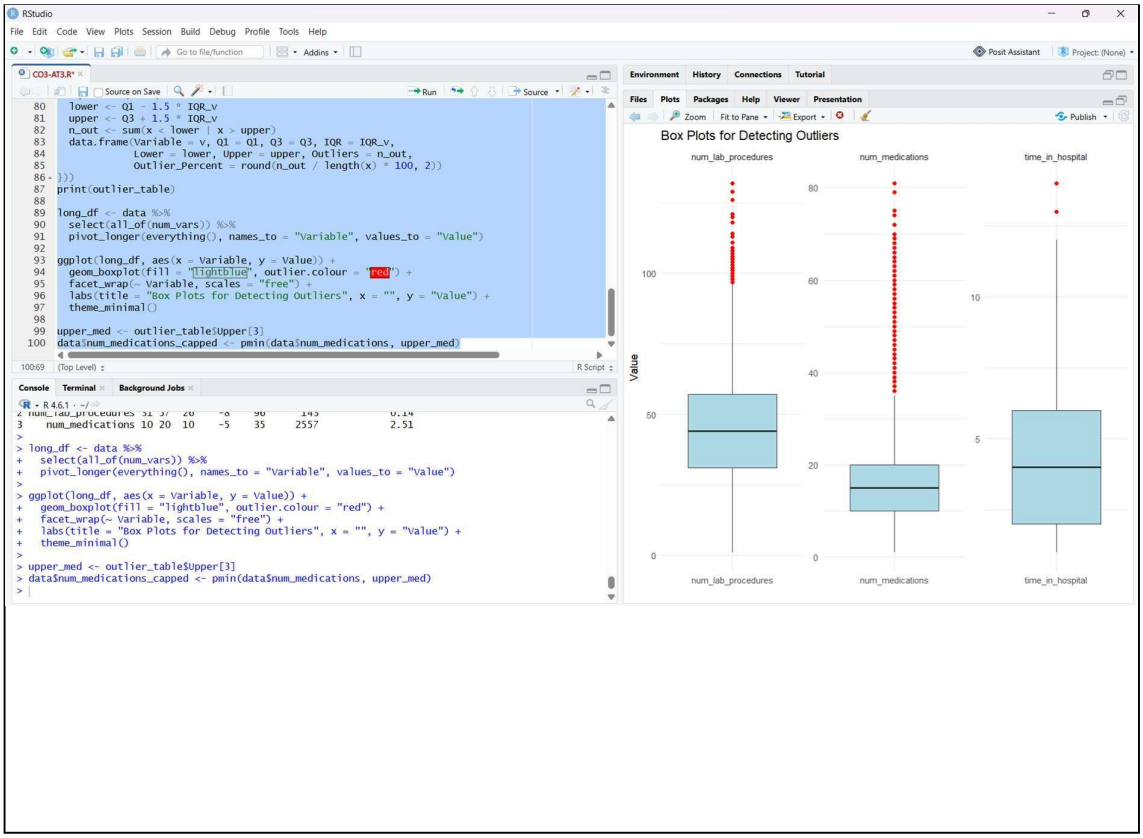
ggplot(long_df, aes(x = Variable, y = Value)) +
  geom_boxplot(fill = "lightblue", outlier.colour = "red") +
  facet_wrap(~ Variable, scales = "free") +
  labs(title = "Box Plots for Detecting Outliers", x = "", y = "Value") +
  theme_minimal()

# Treatment: cap extreme values at the upper fence (winsorizing)
upper_med <- outlier_table$Upper[3]   # num_medications
data$num_medications_capped <- pmin(data$num_medications, upper_med)
```

Graph for Visualization

Graph type: Box plot (three panels). The box shows Q1 to Q3, the line inside it is the median, and the individual points beyond the whiskers are outliers.

Output


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
CO3-AT3.R*
Source on Save
Run
Source
lower <- Q1 - 1.5 * IQR_v
upper <- Q3 + 1.5 * IQR_v
n_out <- sum(x < lower | x > upper)
Lower = lower, Upper = upper, Outliers = n_out,
Outlier_Percent = round(n_out / length(x) * 100, 2))
}))
print(outlier_table)
long_df <- data %>%
select(all_of(num_vars)) %>%
pivot_longer(everything(), names_to = "Variable", values_to = "Value")
ggplot(long_df, aes(x = Variable, y = Value)) +
geom_boxplot(fill = "lightblue", outlier.colour = "red") +
facet_wrap(~ Variable, scales = "free") +
labs(title = "Box Plots for Detecting Outliers", x = "", y = "Value") +
theme_minimal()
upper_med <- outlier_table$Upper[3]
data$num_medications_capped <- pmin(data$num_medications, upper_med)
Console
Terminal
Background Jobs
R 4.6.1 · ~/
3 num_medications 10 20 10 -5 35 2557 2.51
> long_df <- data %>%
+ select(all_of(num_vars)) %>%
+ pivot_longer(everything(), names_to = "Variable", values_to = "Value")
> ggplot(long_df, aes(x = Variable, y = Value)) +
+ geom_boxplot(fill = "lightblue", outlier.colour = "red") +
+ facet_wrap(~ Variable, scales = "free") +
+ labs(title = "Box Plots for Detecting Outliers", x = "", y = "Value") +
+ theme_minimal()
> upper_med <- outlier_table$Upper[3]
> data$num_medications_capped <- pmin(data$num_medications, upper_med)
Environment
History
Connections
Tutorial
Files
Plots
Packages
Help
Viewer
Presentation
Zoom
Fit to Pane
Export
Publish
Box Plots for Detecting Outliers
num_lab_procedures
num_medications
time_in_hospital
Value
0
20
40
60
80
50
100
5
10

Question-4: How can the readmission attribute be converted into a binary target variable, and how balanced are the resulting classes?

Formula Used and Explanation

Binary Target: y = 1 if readmitted = "<30", otherwise y = 0

Readmission Rate (%) = (Number of patients with y = 1 ÷ Total patients) × 100

Imbalance Ratio = Number in majority class ÷ Number in minority class

The original attribute readmitted has three values: NO (not readmitted), >30 (readmitted after 30 days) and <30 (readmitted within 30 days). Since the objective is to identify factors linked to early readmission, the attribute is encoded as a binary variable that equals 1 only for readmission within 30 days.

The program also calculates the readmission rate and the imbalance ratio. A high imbalance ratio shows that early readmissions are the minority class, which must be considered later during model building and evaluation (for example by using resampling or class weights).

A pie chart is used because it shows the share of each readmission category as part of the whole dataset.

R Program

```
# Q4 - Target variable creation and class distribution
library(dplyr)
library(ggplot2)

data <- read.csv("diabetic_data.csv", na.strings = c("?", ""),
                 stringsAsFactors = FALSE)

# Binary target: 1 = readmitted within 30 days, 0 = otherwise
data$readmit_30 <- ifelse(data$readmitted == "<30", 1, 0)

dist <- data %>%
  count(readmitted) %>%
  mutate(Percent = round(n / sum(n) * 100, 1),
         Label = paste0(readmitted, " (", Percent, "%)"))
print(dist)

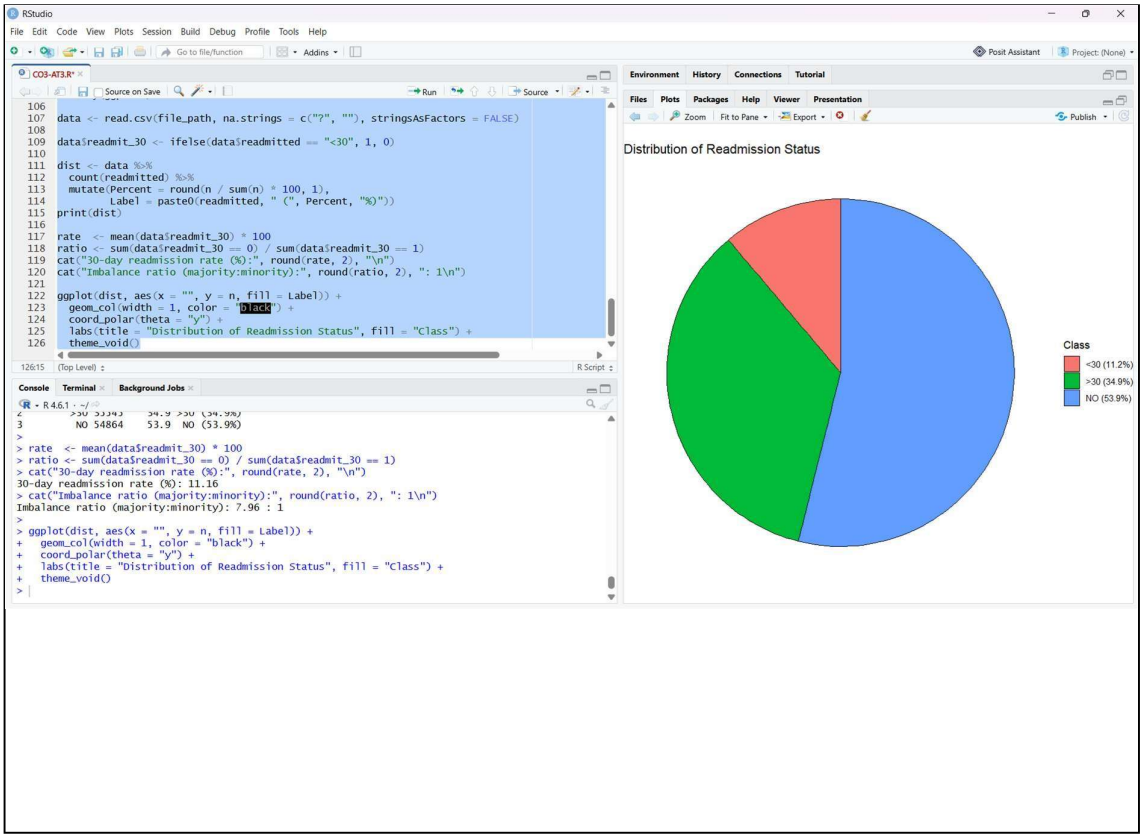
rate  <- mean(data$readmit_30) * 100
ratio <- sum(data$readmit_30 == 0) / sum(data$readmit_30 == 1)
cat("30-day readmission rate (%):", round(rate, 2), "\n")
cat("Imbalance ratio (majority:minority):", round(ratio, 2), ": 1\n")

# Graph: pie chart
ggplot(dist, aes(x = "", y = n, fill = Label)) +
  geom_col(width = 1, color = "black") +
  coord_polar(theta = "y") +
  labs(title = "Distribution of Readmission Status", fill = "Class") +
  theme_void()
```

Graph for Visualization

Graph type: Pie chart. Each slice represents one readmission category (NO, >30, <30) with its percentage.

Output


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
CO3-AT3.R*
Source on Save
Run
Source
106
107 data <- read.csv(file_path, na.strings = c("?", ""), stringsAsFactors = FALSE)
108
109 data$readmit_30 <- ifelse(data$readmitted == "<30", 1, 0)
110
111 dist <- data %>%
112 count(readmitted) %>%
113 mutate(Percent = round(n / sum(n) * 100, 1),
114 Label = paste0(readmitted, " (", Percent, "%)"))
115 print(dist)
116
117 rate <- mean(data$readmit_30) * 100
118 ratio <- sum(data$readmit_30 == 0) / sum(data$readmit_30 == 1)
119 cat("30-day readmission rate (%):", round(rate, 2), "\n")
120 cat("Imbalance ratio (majority:minority):", round(ratio, 2), ": 1\n")
121
122 ggplot(dist, aes(x = "", y = n, fill = Label)) +
123 geom_col(width = 1, color = "black") +
124 coord_polar(theta = "y") +
125 labs(title = "Distribution of Readmission Status", fill = "Class") +
126 theme_void()
126:15 (Top Level)
R Script
Console Terminal Background Jobs
R 4.6.1 · ~/
3 NO 54864 53.9 NO (53.9%)
> rate <- mean(data$readmit_30) * 100
> ratio <- sum(data$readmit_30 == 0) / sum(data$readmit_30 == 1)
> cat("30-day readmission rate (%):", round(rate, 2), "\n")
30-day readmission rate (%): 11.16
> cat("Imbalance ratio (majority:minority):", round(ratio, 2), ": 1\n")
Imbalance ratio (majority:minority): 7.96 : 1
> ggplot(dist, aes(x = "", y = n, fill = Label)) +
+ geom_col(width = 1, color = "black") +
+ coord_polar(theta = "y") +
+ labs(title = "Distribution of Readmission Status", fill = "Class") +
+ theme_void()
Environment History Connections Tutorial
Files Plots Packages Help Viewer Presentation
Zoom Fit to Pane Export Publish
Distribution of Readmission Status
Class
<30 (11.2%)
>30 (34.9%)
NO (53.9%)

Question-5: How can the numerical attributes be scaled using Min-Max normalization and Z-score standardization, and how are two scaled variables related?

Formula Used and Explanation

$$\textit{Min-Max Normalization: } x' = (x - x_min) \div (x_max - x_min)$$

$$\textit{Z-score Standardization: } z = (x - \mu) \div \sigma$$

$$\textit{where } \mu = \textit{mean and } \sigma = \textit{standard deviation of the attribute}$$

Numerical attributes such as time_in_hospital, num_lab_procedures and num_medications are measured on very different scales. Min-Max normalization rescales each attribute to the range 0 to 1, while Z-score standardization rescales it to a mean of 0 and a standard deviation of 1. Scaling prevents attributes with large values from dominating distance-based and gradient-based algorithms.

The program applies both methods and verifies the result: Min-Max values must lie between 0 and 1, and Z-score columns must have a mean close to 0 and a standard deviation of 1. A scatter plot of two standardized variables with a regression line then shows how they are related after scaling.

R Program

```
# Q5 - Feature scaling (Min-Max and Z-score)
library(dplyr)
library(ggplot2)

data <- read.csv("diabetic_data.csv", na.strings = c("?", ""),
                 stringsAsFactors = FALSE)
num_vars <- c("time_in_hospital", "num_lab_procedures",
              "num_procedures", "num_medications", "number_diagnoses")

minmax <- function(x) (x - min(x)) / (max(x) - min(x))

scaled_minmax <- data %>% mutate(across(all_of(num_vars), minmax))
scaled_z      <- data %>% mutate(across(all_of(num_vars),
                                        ~ as.numeric(scale(.x))))

# Verification
print(summary(scaled_minmax[num_vars]))           # min = 0, max = 1
print(round(colMeans(scaled_z[num_vars]),  3))    # mean = 0
print(round(apply(scaled_z[num_vars], 2, sd), 3))# sd = 1

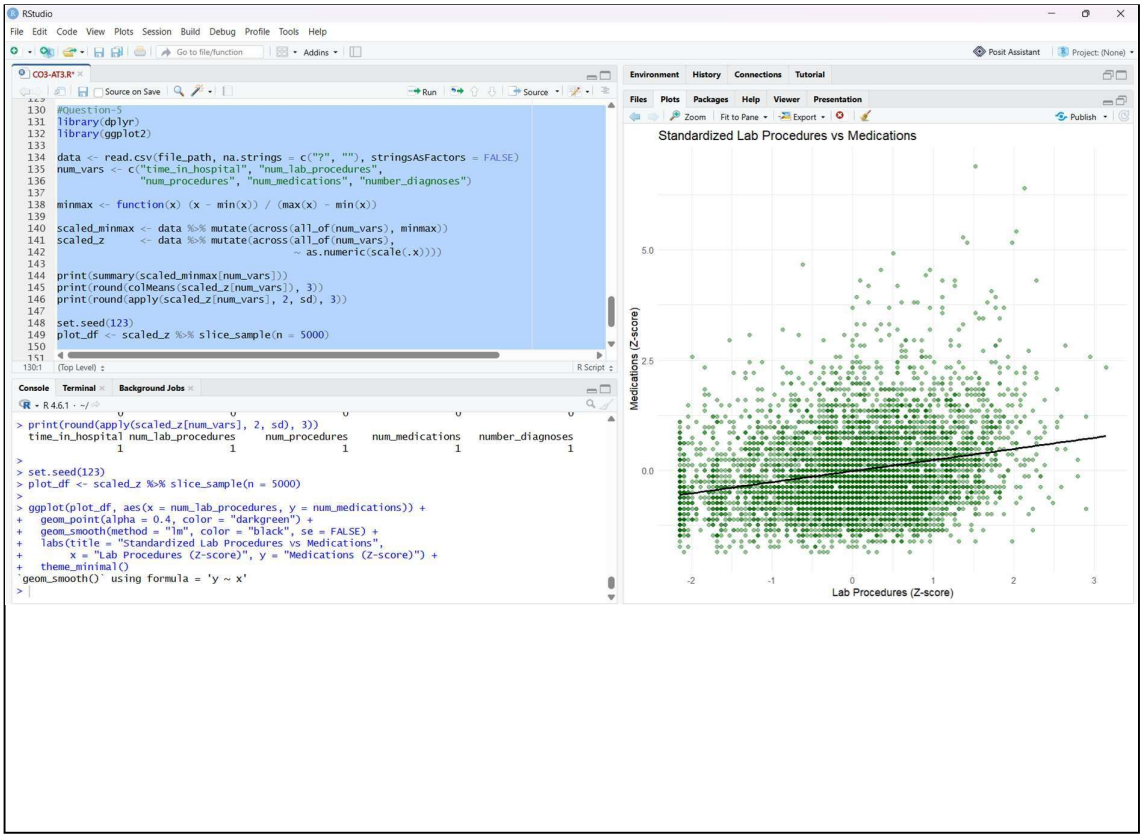
# Graph: scatter plot with regression line (5000-record sample)
set.seed(123)
plot_df <- scaled_z %>% slice_sample(n = 5000)

ggplot(plot_df, aes(x = num_lab_procedures, y = num_medications)) +
  geom_point(alpha = 0.4, color = "darkgreen") +
  geom_smooth(method = "lm", color = "black", se = FALSE) +
  labs(title = "Standardized Lab Procedures vs Medications",
       x = "Lab Procedures (Z-score)", y = "Medications (Z-score)") +
  theme_minimal()
```

Graph for Visualization

Graph type: Scatter plot with a regression line, drawn on a random sample of 5,000 records for clarity.

Output


RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
CO3-AT3.R*
#Question-5
library(dplyr)
library(ggplot2)
data <- read.csv(file_path, na.strings = c("?", ""), stringsAsFactors = FALSE)
num_vars <- c("time_in_hospital", "num_lab_procedures",
"num_procedures", "num_medications", "number_diagnoses")
minmax <- function(x) (x - min(x)) / (max(x) - min(x))
scaled_minmax <- data %>% mutate(across(all_of(num_vars), minmax))
scaled_z <- data %>% mutate(across(all_of(num_vars),
~ as.numeric(scale(.x))))
print(summary(scaled_minmax[num_vars]))
print(round(colMeans(scaled_z[num_vars]), 3))
print(round(apply(scaled_z[num_vars], 2, sd), 3))
set.seed(123)
plot_df <- scaled_z %>% slice_sample(n = 5000)
Console Terminal Background Jobs
> print(round(apply(scaled_z[num_vars], 2, sd), 3))
time_in_hospital num_lab_procedures num_procedures num_medications number_diagnoses
1 1 1 1 1
> set.seed(123)
> plot_df <- scaled_z %>% slice_sample(n = 5000)
> ggplot(plot_df, aes(x = num_lab_procedures, y = num_medications)) +
+ geom_point(alpha = 0.4, color = "darkgreen") +
+ geom_smooth(method = "lm", color = "black", se = FALSE) +
+ labs(title = "Standardized Lab Procedures vs Medications",
+ x = "Lab Procedures (Z-score)", y = "Medications (Z-score)") +
+ theme_minimal()
`geom_smooth()` using formula = 'y ~ x'
Environment History Connections Tutorial
Files Plots Packages Help Viewer Presentation
Standardized Lab Procedures vs Medications
Medications (Z-score)
Lab Procedures (Z-score)
5.0
2.5
0.0
-2
-1
0
1
2
3